

RECO: Intelligent E-Commerce Platform with Recommendation Engine

Requirements & Initial Design Document

Description

RECO is a full-stack intelligent e-commerce platform designed to simulate a real-world online shopping experience with a built-in recommendation engine. The system addresses the problem of product overload in modern e-commerce platforms by providing users with personalized product suggestions, while also offering a complete end-to-end shopping flow including a mock payment system.

Problem Statement

Modern e-commerce platforms contain massive product catalogs, which often lead to:

- Decision fatigue
- Abandoned shopping carts
- Difficulty finding relevant products

Additionally, real payment integrations are difficult to implement in academic environments.

Solution

RECO solves these challenges by providing:

- A **self-contained e-commerce system**
- A **mock payment gateway** for simulated checkout
- A **data-driven recommendation engine** that adapts to user behavior

Target Users

- **Customers:** Browse and search products, manage cart and orders, receive personalized recommendations, complete simulated checkout process
 - **Administrators:** Manage product catalog (CRUD), monitor orders, control inventory system
-

Functional Requirements

FR-1: User Authentication & Authorization

- FR-1.1: Users shall be able to register with email, password, and profile information
- FR-1.2: Users shall be able to log in using JWT-based authentication
- FR-1.3: System shall support role-based access control (Admin / Customer)
- FR-1.4: Users shall be able to update their profile information

- FR-1.5: System shall securely store passwords using hashing
- FR-1.6: Users shall be able to delete their accounts

FR-2: Product Catalog Management

- FR-2.1: Admins shall be able to create, read, update, and delete products
- FR-2.2: Products shall include name, description, price, category, image, and stock quantity
- FR-2.3: Customers shall be able to browse and search products by name, category, or tags
- FR-2.4: System shall display product details including recommendations
- FR-2.5: System shall support product pagination and filtering

FR-3: Shopping Cart Functionality

- FR-3.1: Customers shall be able to add products to their shopping cart
- FR-3.2: Customers shall be able to update quantities or remove items from cart
- FR-3.3: System shall calculate cart totals including all items
- FR-3.4: Cart data shall persist across user sessions

FR-4: Order Processing System

- FR-4.1: Customers shall be able to place orders from their cart
- FR-4.2: System shall generate unique order IDs and timestamps
- FR-4.3: System shall track order status (pending, processing, shipped, delivered)
- FR-4.4: System shall notify the user about order stats
- FR-4.5: Customers shall be able to view their order history
- FR-4.6: Admins shall be able to view and manage all orders

FR-5: Mock Payment Gateway

- FR-5.1: System shall simulate a checkout flow (Cart → Payment → Order Confirmation)
- FR-5.2: System shall accept mock payment information
- FR-5.3: System shall generate payment records with transaction details
- FR-5.4: No external payment APIs shall be required

FR-6: Hybrid Recommendation Engine

- FR-6.1: System shall provide **Cold Start Strategy** recommendations for new users (popular products)
- FR-6.2: System shall provide **Content-Based Filtering** recommendations based on category/tags similarity
- FR-6.3: System shall provide **Collaborative Filtering** recommendations based on "users also bought" patterns
- FR-6.4: Recommendations shall be displayed on product detail pages and user dashboard
- FR-6.5: System shall analyze user behavior "purchases and reviews" to improve recommendations over time

- FR-6.6: System shall use products rated 4+ stars from reviews to find similar products via product similarity scores

FR-7: Product Reviews & Ratings

- FR-7.1: Customers shall be able to create reviews with a rating (1-5) and optional comment for purchased products
 - FR-7.2: System shall automatically update the product's average rating when a review is created, updated, or deleted
 - FR-7.3: Customers shall be able to view all reviews for a product with pagination and sorting (by rating or date)
 - FR-7.4: System shall display a rating distribution breakdown (5-star to 1-star counts) for each product
 - FR-7.5: Customers shall be able to update or delete their own reviews
 - FR-7.6: System shall allow customers to view all reviews they have written across products
-

User Stories / Use Cases

UC-1: Customer Registration & Login

Actor: Customer

Description: As a new user, I want to create an account so that I can shop and receive personalized recommendations.

Flow:

1. User navigates to registration page
2. User enters email, password, and name
3. System validates input and creates account
4. User receives confirmation and can log in
5. System generates JWT token for authentication

UC-2: Browse & Search Products

Actor: Customer

Description: As a customer, I want to browse and search products so that I can find items I'm interested in.

Flow:

1. User views product catalog (paginated)
2. User searches by keyword, category, or filters by price range
3. System displays matching products with images, prices, and ratings
4. User clicks on a product to view details and recommendations

UC-3: Add to Cart & Checkout

Actor: Customer

Description: As a customer, I want to add items to my cart and complete a purchase so that I can receive my order.

Flow:

1. User clicks "Add to Cart" on a product
2. System updates cart and shows confirmation
3. User proceeds to checkout
4. User enters mock payment information
5. System processes mock payment and creates order
6. User receives order confirmation

UC-4: View Personalized Recommendations

Actor: Customer

Description: As a customer, I want to see product recommendations so that I can discover items I might like.

Flow:

1. User views product detail page or dashboard
2. System analyzes user's purchase history and behavior
3. System displays recommended products based on hybrid strategy
4. User can click on recommendations to add to cart or view details

UC-5: Admin Product Management

Actor: Administrator

Description: As an admin, I want to manage the product catalog so that I can keep inventory up to date.

Flow:

1. Admin logs into admin dashboard
2. Admin creates/edits/deletes products
3. System updates database and reflects changes immediately
4. Admin can view stock levels and manage inventory

UC-6: Admin Order Monitoring

Actor: Administrator

Description: As an admin, I want to monitor and manage orders so that I can fulfill them efficiently.

Flow:

1. Admin views all orders in dashboard
2. Admin can filter by status, date, or customer
3. System notifies customer of status changes

UC-7: Write & Manage Product Reviews

Actor: Customer

Description: As a customer, I want to leave reviews and ratings for products I purchased so that I can share my experience and help other shoppers.

Flow:

1. User navigates to a product detail page
 2. User clicks "Write Review" and selects a rating (1-5) with an optional comment
 3. System validates input and creates the review record
 4. System recalculates the product's average rating and rating distribution
 5. User can later edit or delete their own review
 6. Other customers can view all reviews sorted by rating or date
-

Non-Functional Requirements

NFR-1: Performance

- NFR-1.1: API response time shall be under 200ms for standard requests
- NFR-1.2: Recommendation engine shall generate suggestions within 500ms
- NFR-1.3: System shall support concurrent users (minimum 100 concurrent sessions)

NFR-2: Security

- NFR-2.1: All passwords shall be hashed using bcrypt or equivalent
- NFR-2.2: JWT tokens shall expire after a configurable timeout (default: 24 hours)
- NFR-2.3: All API endpoints shall enforce role-based access control
- NFR-2.4: Sensitive data shall be transmitted over HTTPS in production

NFR-3: Scalability

- NFR-3.1: System shall be designed with a layered architecture for easy horizontal scaling
- NFR-3.2: Database queries shall be optimized with proper indexing
- NFR-3.3: Recommendation engine shall use SQL-based logic for efficient computation

NFR-4: Availability

- NFR-4.1: System shall maintain 99.5% uptime during business hours
- NFR-4.2: Database shall implement automatic backup procedures

NFR-5: Usability

- NFR-5.1: Frontend shall be responsive and mobile-friendly
- NFR-5.2: UI shall follow consistent design patterns
- NFR-5.3: Error messages shall be clear and actionable

NFR-6: Maintainability

- NFR-6.1: Code shall follow clean architecture principles (Controller → Service → Repository)
- NFR-6.2: API shall be fully documented using Swagger
- NFR-6.3: Unit tests shall cover critical business logic

Constraints & Assumptions

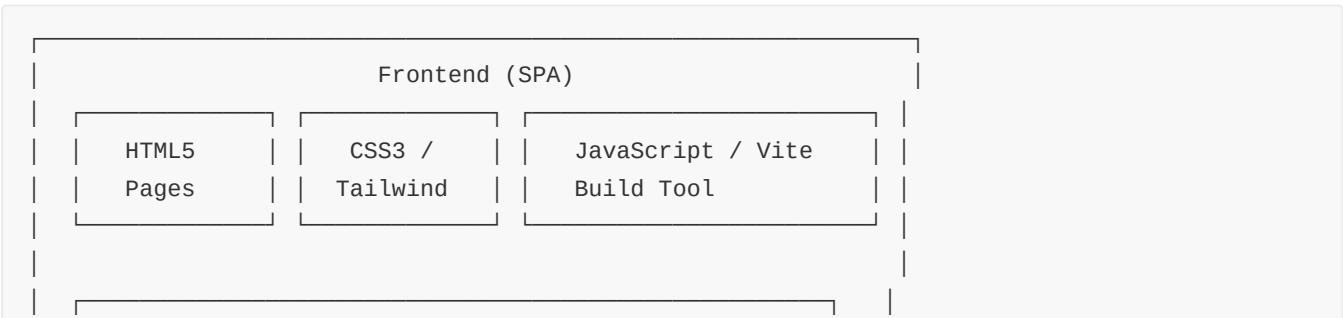
Constraints

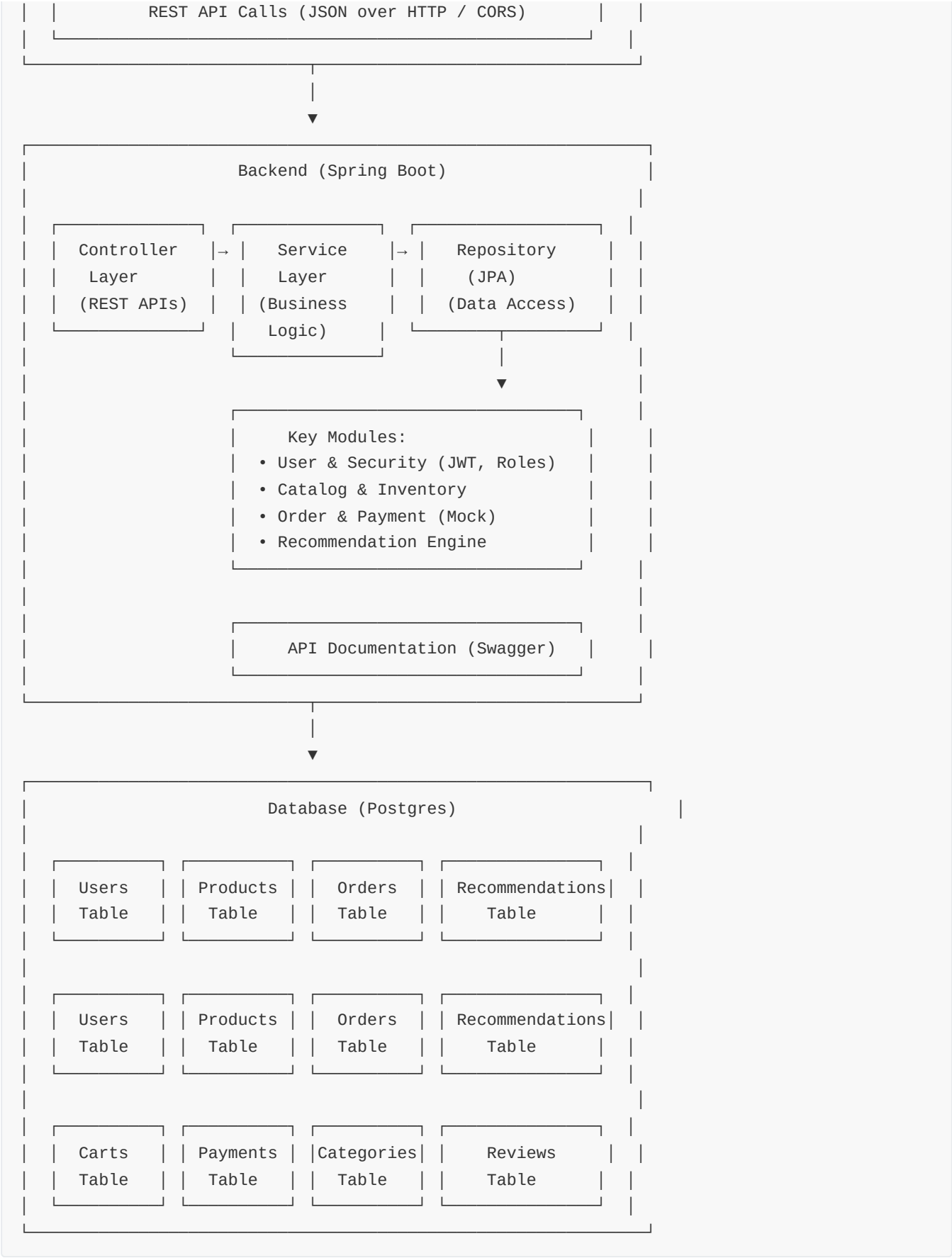
- C-1: **Academic Environment:** Real payment gateway integration is not required; mock payment system suffices
- C-2: **Tech Stack:** Backend must use Java/Spring Boot; Frontend must use HTML/CSS/JavaScript with Tailwind CSS and Vite
- C-3: **Database:** Postgres is the designated relational database
- C-4: **Recommendation Engine:** Must use SQL-based recommendation logic (no external ML services)

Assumptions

- A-1: Users have modern web browsers with JavaScript enabled
- A-2: Product images and descriptions are provided by administrators
- A-3: Mock payment data does not need to be validated against real financial systems
- A-4: User behavior data (purchases, views) will be sufficient for recommendation accuracy
- A-5: System will start with a limited product catalog that can grow over time
- A-6: CORS is properly configured for frontend-backend communication during development
- A-7: Recommendation strategies (Cold Start, Content-Based, Collaborative) will be selected dynamically based on user data availability

High-Level Architecture





Architecture Layers

Layer	Responsibility
Frontend (SPA)	User interface, routing, state management, API communication
Controller Layer	REST API endpoints, request validation, response formatting
Service Layer	Business logic, recommendation strategy selection, payment processing
Repository Layer	Database operations via Spring Data JPA
Entity Layer	JPA entity mappings to Postgres tables

Selected Design Patterns

DP-1: Factory Pattern

Purpose: Dynamically select recommendation strategies based on user context

Justification: The hybrid recommendation engine needs to choose between three different strategies (Cold Start, Content-Based, Collaborative Filtering) depending on available user data. The Factory Pattern encapsulates this selection logic, making it easy to add new strategies in the future without modifying existing code.

Implementation:

- `RecommendationFactory` creates the appropriate strategy instance
- `ColdStartStrategy`: Returns popular products for new users with no history
- `ContentBasedStrategy`: Returns similar products based on category/tags
- Factory evaluates user data availability and selects the optimal strategy

Benefits:

- Open/Closed Principle: New strategies can be added without changing factory logic
- Single Responsibility: Each strategy handles its own recommendation logic
- Easy testing: Each strategy can be unit tested independently

DP-2: Builder Pattern

Purpose: Construct complex objects like `Order` and `Product` with multiple optional fields

Justification: Entities like `Order` and `Product` have many fields (some required, some optional). Using Lombok's `@Builder` annotation provides a fluent API for constructing these objects, improving code readability and reducing constructor complexity.

Implementation:


```
@Builder
public class Order {
    private Long id;
    private User customer;
    private List<OrderItem> items;
    private Payment payment;
    private OrderStatus status;
    private LocalDateTime createdAt;
    // ...
}
```

Benefits:

- Readable object construction: `Order.builder().customer(user).items(items).build()`
- Immutability: Built objects can be made immutable
- Type safety: Compile-time checking of required fields
- Default values: Optional fields can have sensible defaults

DP-3: Singleton Pattern

Purpose: Ensure single instances of core services like `MockPaymentGateway` and `RecommendationService`

Justification: The mock payment gateway and recommendation service are shared resources that should have a single instance across the application. This ensures consistent state, reduces memory overhead, and provides a centralized point for configuration and logging.

Implementation:

- `MockPaymentGateway`: Single instance handles all mock payment transactions
- `RecommendationService`: Single instance manages recommendation strategy selection and caching
- Spring's `@Service` annotation provides singleton behavior by default
- Thread-safe: Spring manages singleton instances in a thread-safe manner

Benefits:

- Controlled access: Single point of access to shared resources
- Resource efficiency: Reduces memory and initialization overhead
- Consistent state: All components use the same instance
- Easy configuration: Centralized configuration and logging

Initial Class List

Core Domain Classes

Class Name	Responsibility	Key Attributes	Key Methods
User	Represents application users	<code>id</code> , <code>email</code> , <code>password</code> , <code>name</code> , <code>role</code> , <code>createdAt</code>	<code>register()</code> , <code>login()</code> , <code>updateProfile()</code>

Class Name	Responsibility	Key Attributes	Key Methods
Product	Represents catalog items	id, name, description, price, category, tags, imageUrl, stockQuantity	create(), update(), delete(), search()
Category	Product categorization	id, name, description, parentCategory	create(), getProducts()
Cart	Shopping cart for user	id, userId, items, totalAmount, createdAt	addItem(), removeItem(), updateQuantity(), checkout()
CartItem	Individual cart line item	id, cartId, productId, quantity, price	calculateSubtotal()
Order	Customer purchase record	id, userId, items, totalAmount, status, createdAt, paymentId	create(), updateStatus(), getDetails()
OrderItem	Individual order line item	id, orderId, productId, quantity, price	calculateSubtotal()
Payment	Payment transaction record	id, orderId, amount, status, transactionId, timestamp	processPayment()
Review	Product review and rating	id, userId, productId, rating, comment, createdAt, updatedAt	create(), update(), delete(), calculateAverageRating()
Recommendation	Product recommendation data	id, userId, recommendedProducts, strategy, generatedAt	generate(), getRecommendations()

Service Layer Classes

Class Name	Responsibility	Key Methods
UserService	User authentication and profile management	register(), authenticate(), generateToken(), updateProfile()
ProductService	Product catalog CRUD operations	createProduct(), updateProduct(), deleteProduct(), searchProducts(), getProductById()
CartService	Shopping cart operations	addToCart(), removeFromCart(), updateQuantity(), getCart(), calculateTotal()
OrderService	Order processing and management	createOrder(), getOrdersByUser(), updateOrderStatus(), getAllOrders()
PaymentService	Mock payment processing	processPayment(), refundPayment(), getPaymentDetails()

Class Name	Responsibility	Key Methods
ReviewService	Review and rating management	<code>createReview()</code> , <code>updateReview()</code> , <code>deleteReview()</code> , <code>getProductReviews()</code> , <code>getUserReviews()</code> , <code>calculateAverageRating()</code>
RecommendationService	Hybrid recommendation engine	<code>getRecommendations()</code> , <code>selectStrategy()</code> , <code>analyzeUserBehavior()</code>

Recommendation Strategy Classes

Class Name	Responsibility	Key Methods
RecommendationStrategy (Interface)	Base strategy interface	<code>getRecommendations(userId, limit)</code>
ColdStartStrategy	Popular products for new users	<code>getPopularProducts()</code> , <code>getTrendingProducts()</code>
ContentBasedStrategy	Similar products by category/tags	<code>getSimilarProducts(productId)</code> , <code>getProductsByCategory(category)</code>
CollaborativeFilteringStrategy	"Users also bought" recommendations	<code>getUsersWhoBoughtThisAlsoBought()</code> , <code>getUserPurchaseHistory()</code>
RecommendationFactory	Strategy selection factory	<code>createStrategy(user)</code>

Controller Layer Classes

Class Name	Responsibility	Key Endpoints
AuthController	Authentication endpoints	<code>POST /api/auth/register</code> , <code>POST /api/auth/login</code>
ProductController	Product catalog endpoints	<code>GET /api/products</code> , <code>GET /api/products/{id}</code> , <code>POST /api/products</code> , <code>PUT /api/products/{id}</code> , <code>DELETE /api/products/{id}</code>
CartController	Cart management endpoints	<code>GET /api/cart</code> , <code>POST /api/cart/items</code> , <code>PUT /api/cart/items/{id}</code> , <code>DELETE /api/cart/items/{id}</code>
OrderController	Order processing endpoints	<code>POST /api/orders</code> , <code>GET /api/orders</code> , <code>GET /api/orders/{id}</code>
PaymentController	Payment endpoints	<code>POST /api/payments</code> , <code>GET /api/payments/{id}</code>

Class Name	Responsibility	Key Endpoints
ReviewController	Review endpoints	POST /api/products/{id}/reviews, GET /api/products/{id}/reviews, GET /api/users/{id}/reviews, PUT /api/reviews/{id}, DELETE /api/reviews/{id}
RecommendationController	Recommendation endpoints	GET /api/recommendations, GET /api/recommendations/product/{id}

Configuration & Utility Classes

Class Name	Responsibility
SecurityConfig	Spring Security and JWT configuration
CorsConfig	CORS configuration for frontend-backend communication
MockPaymentGateway	Singleton mock payment processor
JwtTokenUtil	JWT token generation and validation
ApiResponse	Standardized API response wrapper

Entity Relationships

- User (1) → (N) Cart
- User (1) → (N) Order
- Order (1) → (N) OrderItem
- Product (1) → (N) OrderItem
- Cart (1) → (N) CartItem
- Product (1) → (N) CartItem
- Order (1) → (1) Payment
- Category (1) → (N) Product
- User (1) → (N) Recommendation
- User (1) → (N) Review
- Product (1) → (N) Review

Endpoint Summary

Module	Endpoint	Method	Description
Auth	/api/auth/register	POST	Register new user
Auth	/api/auth/login	POST	User login

Module	Endpoint	Method	Description
Products	/api/products	GET	List all products
Products	/api/products/{id}	GET	Get product details
Products	/api/products	POST	Create product (Admin)
Products	/api/products/{id}	PUT	Update product (Admin)
Products	/api/products/{id}	DELETE	Delete product (Admin)
Cart	/api/cart	GET	Get user cart
Cart	/api/cart/items	POST	Add item to cart
Cart	/api/cart/items/{id}	PUT	Update cart item
Cart	/api/cart/items/{id}	DELETE	Remove cart item
Orders	/api/orders	POST	Create order (checkout)
Orders	/api/orders	GET	Get user orders
Orders	/api/orders/{id}	GET	Get order details
Payments	/api/payments	POST	Process mock payment
Payments	/api/payments/{id}	GET	Get payment details
Recommendations	/api/recommendations	GET	Get user recommendations
Recommendations	/api/recommendations/product/{id}	GET	Get product-based recommendations
Reviews	/api/products/{id}/reviews	POST	Create review for product
Reviews	/api/products/{id}/reviews	GET	Get all reviews for product
Reviews	/api/users/{id}/reviews	GET	Get all reviews by user
Reviews	/api/reviews/{id}	PUT	Update own review
Reviews	/api/reviews/{id}	DELETE	Delete own review
Reviews	/api/products/{id}/rating	GET	Get product rating summary

Technology Stack

Layer	Technology
Backend	Java, Spring Boot, Spring Data JPA, Spring Security
Database	Postgres
Frontend	HTML5, CSS3, JavaScript, Tailwind CSS, Vite

Layer	Technology
Authentication	JWT (JSON Web Tokens)
API Documentation	Swagger / OpenAPI
Testing	Postman
Build Tool	Maven (Backend), Vite (Frontend)

Team Members

- Abdelrahman Hany Farouk (SE1)
- Ahmad Louay Mahmoud (SE1)
- Youssef Waleed (SE1)
- Youssef Khaled (SE1)